

Operands location in Register Memory Immediate value

RISC-V design Principles

- simplicity
- ① consistent instruction format
 - ② same no. of operands
[2 source operands
1 destination operand]
 - ③ easier to Encode/decode and handle in hardware

smaller is faster

RISC-V includes only small no. of Registers ⁽³²⁾

name	Zero	X0	} register notation	→ constant value 0
	ra	X1		→ return address
	sp	X2		→ stack pointer

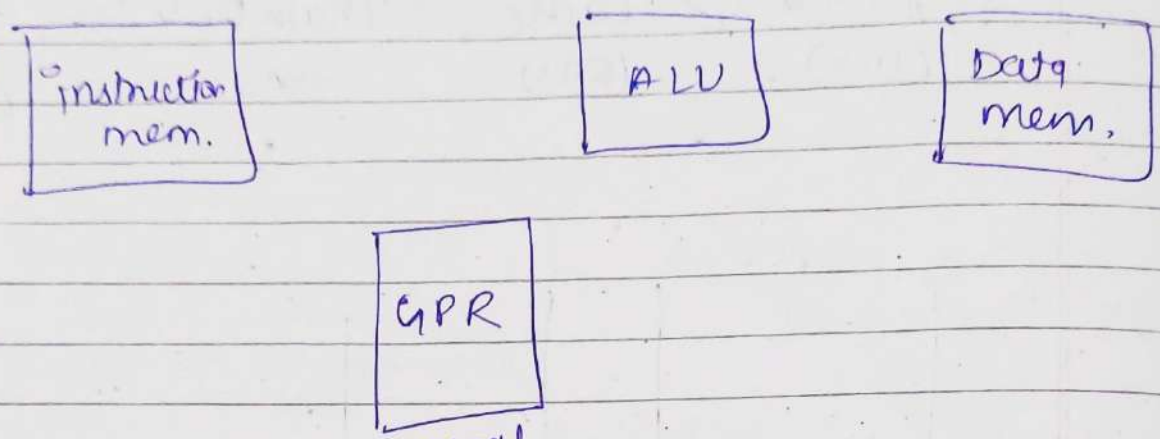
→ RISC-V include commonly used instruction therefore easy to decode, faster

good design demands good compromises

- ① multiple instructions formats allow flexibility
 - ↳ add/sub uses 3 registers
 - ↳ lw/sw uses 2 registers

immediate value avoid extra instructions

controller



general purpose registers

Why needed?

In our microarchitecture

↓
 ALU won't be directly accessing data memory, access only from GPR

- Registers are faster than Memory
- RISC-V is 32 bit Architecture

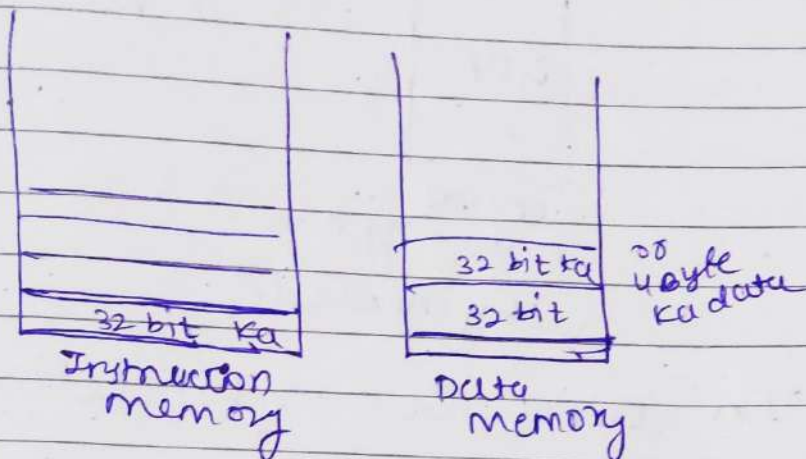
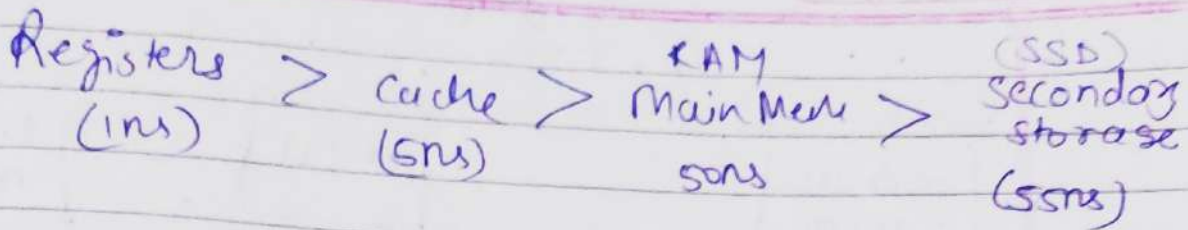
↘ 32 Registers
 ↘ each Register is of 32 bit wide

doubts
 32 bit instruction code
 Instruction mem. → 32 bit wide
 2³² → 4GB locations?
 2¹⁰ = 1024 = 1KB
 2²⁰ = 1024KB
 2³⁰ = 1024MB = 1GB

RISC-V

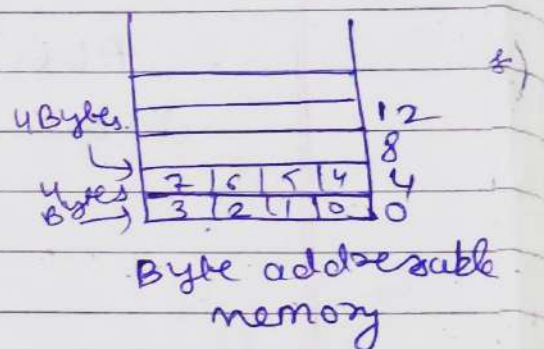
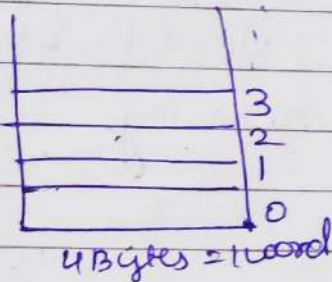
32 bit data

32 bit Instruction (machine code)



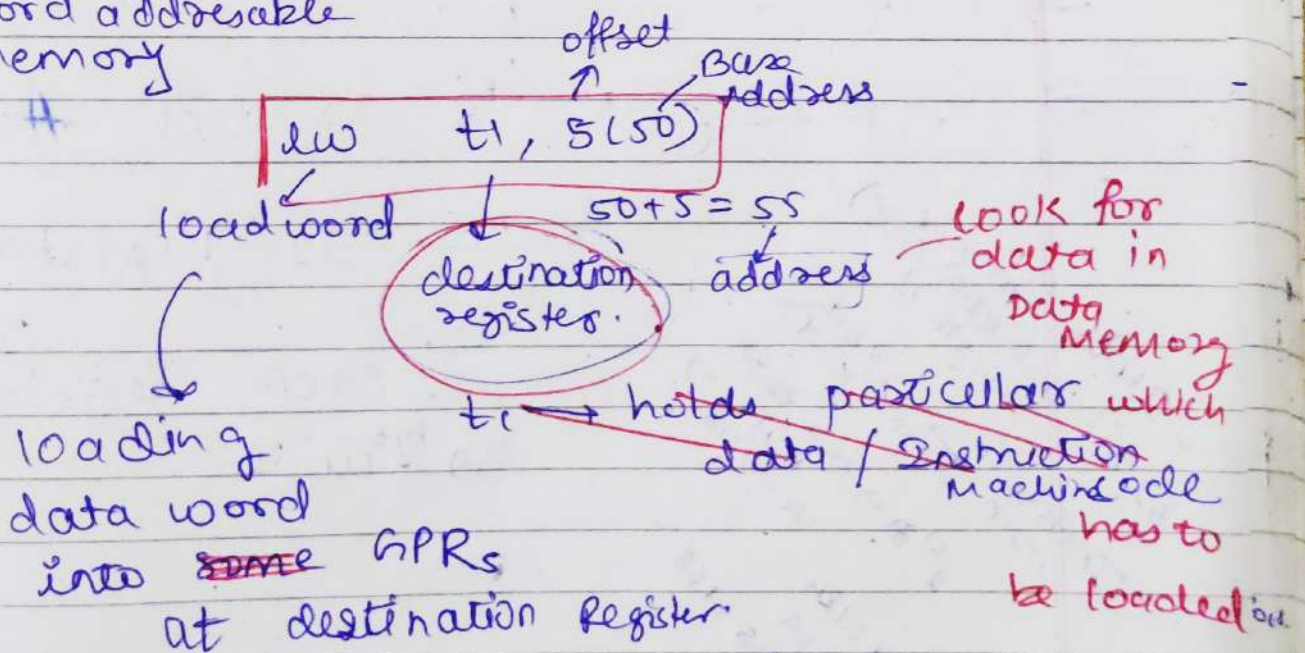
address increases by 4

word addressable memory

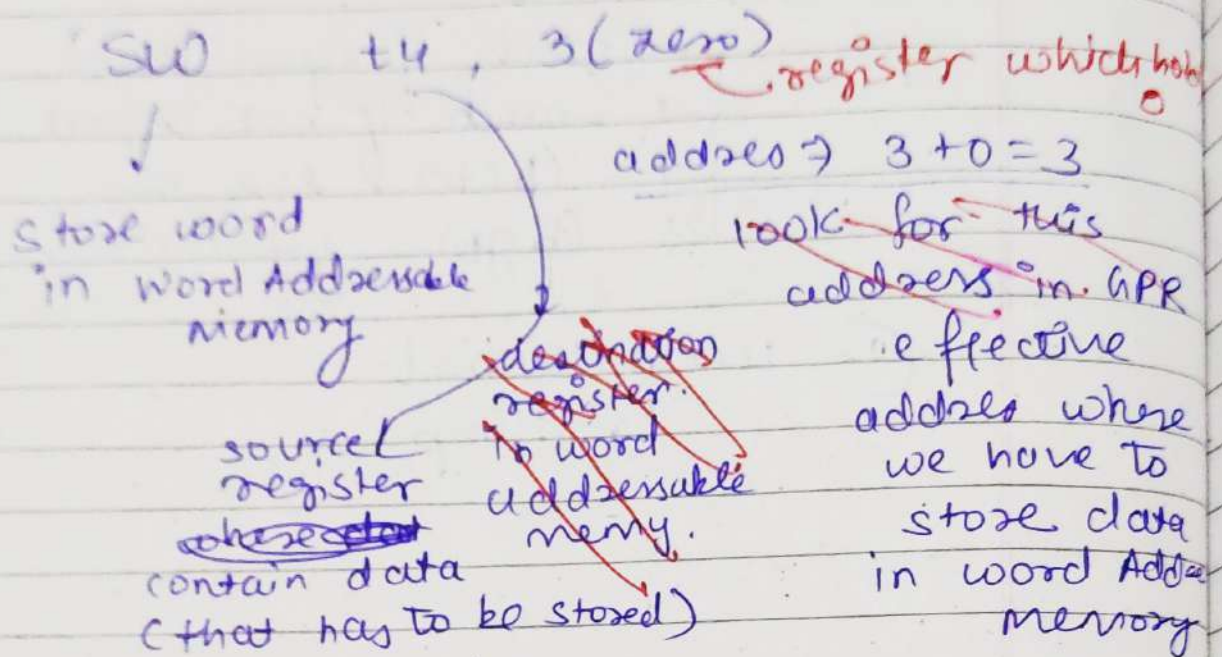


loading from word addressable memory

#



writing in word addressable memory



→ RISC-V ~~is~~ has a Byte Addressable memory

Q control path & Datapath Difference

generates control signals that coordinate these operations by deciding which registers to read,

ALU operation to perform whether to access Mem

performs actual data processing operations such as arithmetic logical mem access register read write

→ control path tells the datapath what to do?

→ datapath does the work

Types of Branches

① Conditional

- ↳ branch if equal (bge)
- ↳ branch if not equal (bne)
- ↳ (blt) less than
- ↳ (bgt) greater than

② unconditional ↳ Jump

eg:-

addi s0, zero, 4

$$\rightarrow s0 = 0 + 4 = 4$$

addi s1, zero, 1

$$s1 = 0 + 1 = 1$$

sll s1, s1, 2

$$s1 = 1 \ll 2 = 4$$

bge s0, s1, target

addi
sub

yx

→ branch is taken

target:

add s0, s1, s0

$$\rightarrow s1 = 5$$

4PR mein hae source register destination register available hai

func 3, func 7, opcode

all three fields used to decide
what operation to perform

Q why r_{s1}, r_{s2}, r_d are 5bits??
2 registers

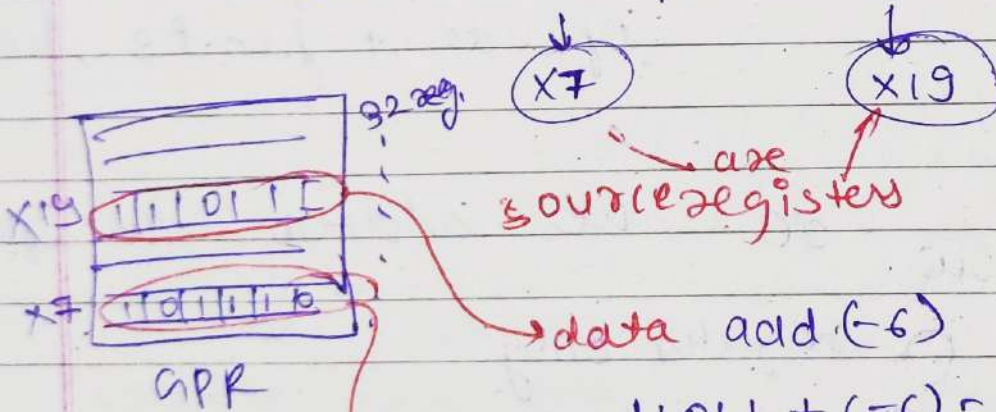
sub to x5, x6, x7 ^{registers}

rd 051 052 assembly point of view

$S = 00101$ $G = 00110$ $F = 00111$

So ~~the~~ To address 32 GPRs
5 bits are used

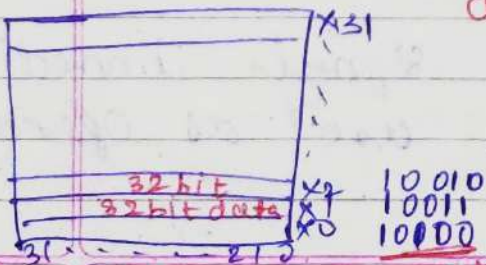
⑤
2 = 32

$$s_{w0} \quad +2, \quad -6(s_3)$$

$$11011 + (-6) = \underline{\underline{10011}}$$

effective
address

data which has to be store in data memory

at effective address (10011)



CPR

5 bit address

Immediate value $\rightarrow$ 160 } uses
SW

addi \$0, \$1, -12 $\rightarrow$ 12' bit in
2's complement
Number

Reading Machine code

$\rightarrow$ written in binary

$\rightarrow$ 0000 1000 000 11000 1000

32 bits

$\rightarrow$ extract the fields
opcode + funct3, funct7

* How do we address the operand?

① register only

$\rightarrow R3 = R1 + R2$

operands found in GPRs

② Immediate

12 bit signed immediate
value used as operand

③ Base Addressing mode

$$\left. \begin{array}{l} \text{lw} \\ \text{sw} \end{array} \right\} \text{Base Address} + \text{immediate} = \text{Address of operand}$$

* 32 bit Instruction

↳ stored in Instruction memory

data is stored in → data Memory

⇒ Program Execution

↳ CPU fetches (reads) instruction from Instruction memory sequence

↳ CPU fast Control Unit decodes the instruction and identifies what operation & operands.

CPU performance

↳ depends on program execution time

$$\text{Execution time} = N \times \text{CPU cycle time} \times \text{CPI}$$

(clock period)

Q 64 Bit RISC-V (RV64Z)

→ No. of Registers (GPRs) = 32

But register ~~can~~ will have operands of 64 bit wide ^{no - x31}

register = 32 registers $\times$ 64 bit = 2048 bit storage

instruction length → 32 bit always $\times$ 64 bit

§ Someone asks related to RISC & CISC

→ then these are architectures

§ How to implement an architecture in hardware?

- 1) single cycle processor
- 2) Multi cycle
- 3) Pipelined

} micro architecture

* § while designing consider subset on RISC-V instructions

R-type } for ALU
I-type }

LD } memory instructions
SW }

Beq } Branch

* Architectural Components

- Memory (data, Instruction)
- ALU
- General Purpose registers (Register file)
- Program Counter

~~ALU~~ all instructions uses
ALU

- ✓ Arithmetic & Logical
- ✓ Lw, Sw, Beqz for effective address calculation
- ✓ Conditional Branching Instructions
 - ↳ A for equality condition

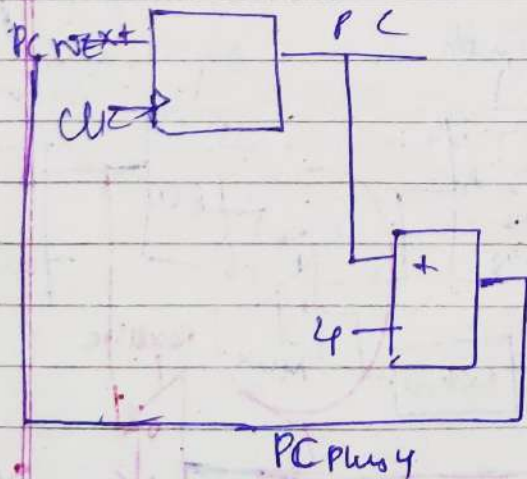
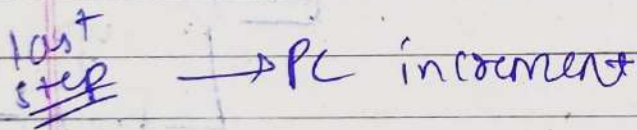
Q Reset is required in a real CPU?
→ yes

PC should be initialized → CPU must know 1st instruction.

Pipeline registers → contain 'X' values

32 bit instruction

imm	rs1	rd	op
12 bit	19:15	func3 11:7	6...0

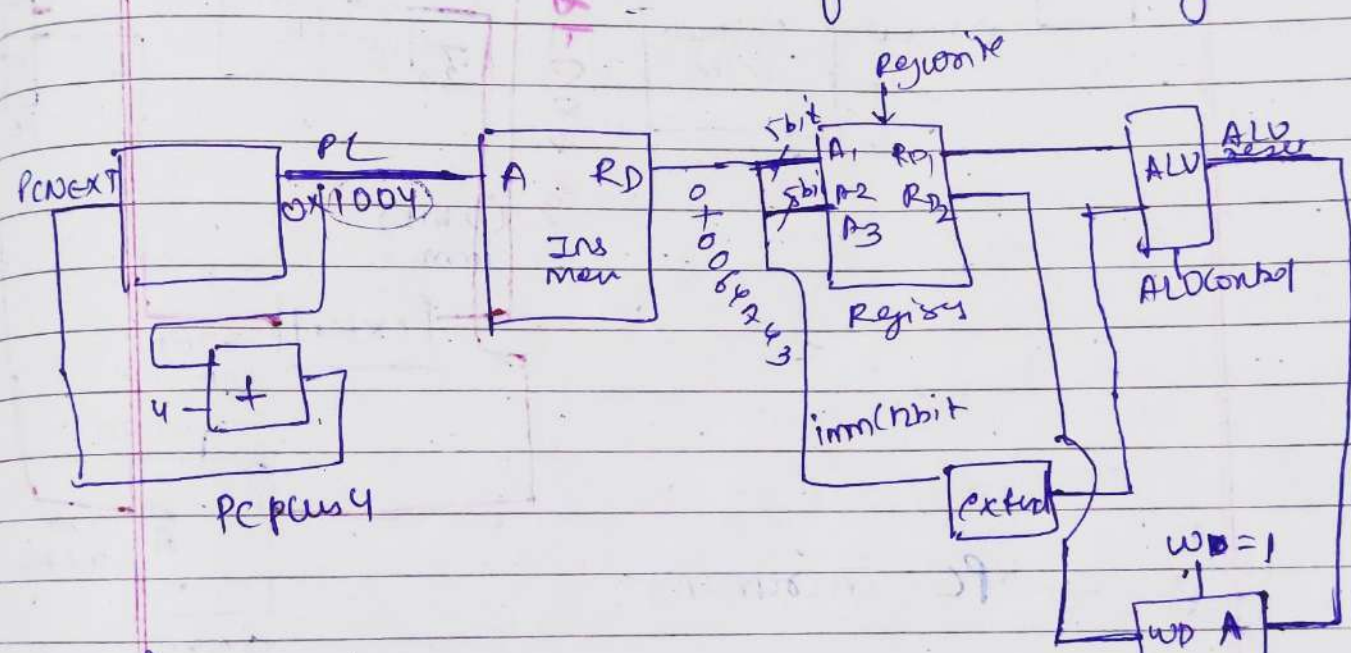


SW
 $X_6, 8(X_9)$
 $rs_2 \quad imm \quad rs_1$

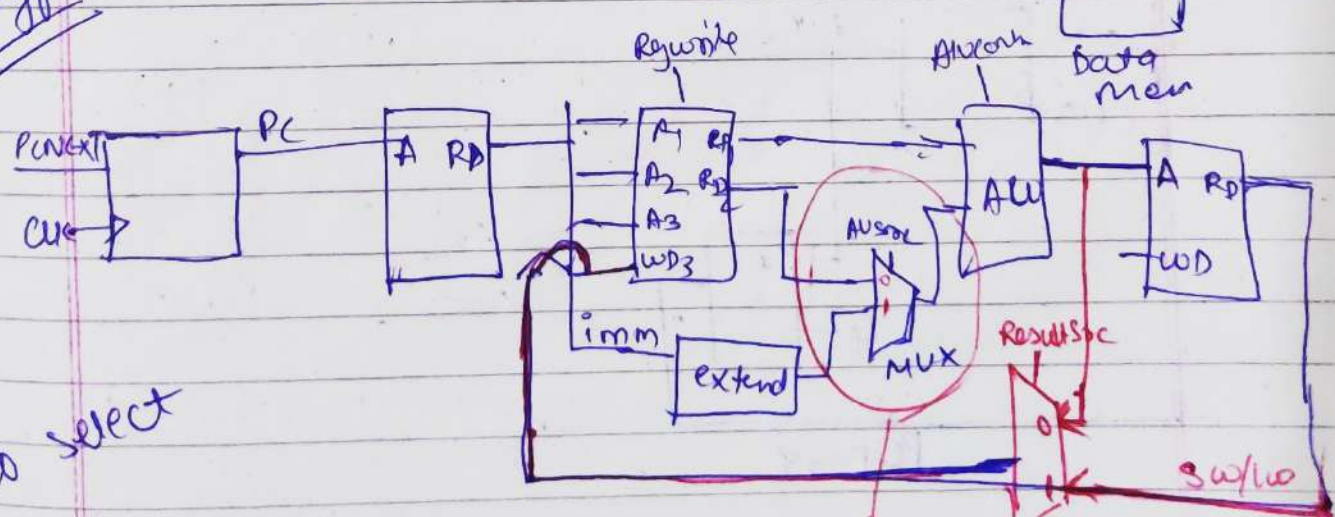


effective add. X_9 (32bit data) + ~~extended~~
 imm

storing data word X_6 which is available X_6 at effective address of data memory



R-type



add rd, rs_1, rs_2
 addi $rd, rs_1, 6$
 BECOZ

SW (Registers to Data Memory)

SW $x_7, -6(x_2)$
 source register effective address

store x_2 into mem $[x_2 - 6]$

IF

ID

source $x_3 = 55$, imm = -6, store $x_7 = 100$

EX

(ALU) $x_3 + (-6) = 55 - 6 = 49$

MEM

data memory access

✓ write opr

✓ MEM[49] = 55

WB

X (nothing)

architecturally store has completed after MEM stage

(Data Mem to Register)

LD $x_5, 8(x_2)$

destination register

cycle by cycle

IF

ID

→ Imm = 8, $x_2 = 100$, destination = x_5

EX

→ ALU effective address $x_2 + 8$

MEM

→ data memory [108] = 25 accessed

WB

→ write result (data = 25) into rd ⇒ x_5

∴ $x_5 = 25$

control hazard

beq s_1, s_2 target

and
sub
add

target :

or

~~soln~~ soln ①

beq

nop
nop

} stalling
the
pipeline

CPU has to
wait
slow

soln ②

Branch Predict
flush

✓ faster than
stalling
but small
penalty

Branch Not taken $(s_1 \neq s_2)$ condition is not satisfied

CPU continues fetching next instruction
beq $\rightarrow$ and $\rightarrow$ sub $\rightarrow$

No flushing of Pipeline needed
No Penalty

Branch is taken (condition true)

CPU jumps to label { once the branch condⁿ is evaluated
CPU already fetch, decode is evaluated

fetch happen before branch condⁿ is evaluated and sub

those instructions must be discarded (flushed)

These wasted instruction cycles are called as Branch misprediction penalty

CPU can predict whether the Branch is taken or not

If CPU prediction → wrong

As CPU already fetch add
sub {flushed
branch evaluated

so that Branch should go to label

CPU is 32 bit

32 bit = 4 bytes

CPU reads 4 bytes at a time.

memory structure

address	Byte
1000	Byte 0
1001	1
1002	2
1003	3

} 4 Bytes
(32 bit word)

1004	4
1005	5
1006	6
1007	7

next 32 bit word

3D	1000	} word aligned addresses every 4 Bytes boundary begins.
	1004	
	1008	
	100C	

→ Example

struct A {

int a;

~~int b;~~ char b;

char c;

};

4 bytes

1 Byte

1 Byte

Total occupied size = $4 + 1 + 1$
= 6 bytes

$\left. \begin{array}{l} 1000 \\ 1001 \\ 1002 \\ 1003 \end{array} \right\}$ int took this much space

structure 1 $\left\{ \begin{array}{l} 1004 \rightarrow \text{char} \\ 1005 \rightarrow \text{char} \end{array} \right.$

structure 2 $\left\{ \begin{array}{l} 1006 \\ 1007 \\ 1008 \\ 1009 \end{array} \right\}$ int

$1006 \% 4 \neq 0$
 Ideally should start at 4 byte aligned address

Therefore structure 2 is misaligned.

$\therefore$ compiler inserts Padding

$\left. \begin{array}{l} 1000 \\ 1001 \\ 1002 \\ 1003 \end{array} \right\}$ int
 1004 char
 1005 char
 1006 } padding
 1007 }

1008 $\rightarrow$ structure 2 begin
 $1008 \% 4 = 0 \checkmark$

misaligned word $\rightarrow$ multiple memory access is required.

hence, $\text{sizeof(struct)} = 8 \text{ Bytes}$

Actual size was 6 Bytes But

2 Bytes added $\rightarrow$ total size 8 bytes

Beoz CPU reads memory most efficiently in 4 Byte word. If

multiple structures are stored in an array, each structure starts on a 4 Byte Boundary.

Q Riscv has Byte addressable memory?

$\rightarrow$ Yes

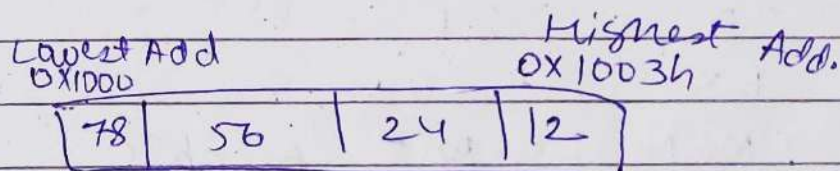
increments by 1	0x1000	$\rightarrow$ 1 Byte (8 bits)	every memory address points to 1 Byte
	0x1001	1 Byte	
	0x1002	1	
	0x1003	1	
	0x1004	1	

suppose memory contains

32 bit value. = ~~22~~ 0x1245678

memory is organized as
Byte stored

0x1000	78 (1 Byte)
0x1001	56 (1 Byte)
1002	24
1003	12



✓ little endian form

Pipeline bubble?

lw $x_1, x_0(2)$
 add $x_1, x_0(2)$
 x_2, x_1, x_4
~~lw~~

empty pipeline
 stage inserted
 due to hazard.

w/o forwarding
 Ins 1

Ins 2

Throughput ↓↓

IF

ID

EX

MEM

WB

IF

ID

-

-

-

Bubble
 (no useful
 work)

waste more
 number of cycles.
 CPI ↑↑

5 stage pipeline

IF - 10ns

ID - 10ns

EX - 10ns

MEM - 10ns

WB - 10ns

✓ balanced pipeline.

Beacaz delay in each
 stage is equal

$$\text{Latency} = 5 \times 10\text{ns} = 50\text{ns}$$

✓ Total time
 to complete
 by 1 instruction
 from IF
 to WB

$$\text{Throughput} = \frac{1}{10\text{ns}}$$

No. of instructions
 completed per
 unit time period = 100 MIPS

uneven pipeline stages affect clock cycle time?

IF - 50ns
ID - 20ns
EX - 60ns
MEM - 10ns

→ If one stage is slower than others
entire pipeline must wait for

it, increases clock period

(reduces throughput)

f_{req} ↓ ↓ (decreases)

$$\text{Throughput} = \frac{1}{60\text{ns}}$$

$$= 16.66 \times 10^6 / \text{sec}$$

if Equal

IF = 20

ID = 20

EX = 20

MEM = 20

$$\rightarrow \text{Throughput} = \frac{1}{20\text{ns}}$$

$$= 50 \times 10^6 / \text{sec}$$

Q Why Pipeline Balancing required?

→ Becoz to reduce clock period
(pipeline cycle time) so that
we can have max operating frequency

→ Throughput to maximize

Q Pipeline has clock period → 10ns
Effective CPI = 1.5

Throughput ??

$$\rightarrow \text{Throughput} = \frac{1}{1.5 \times 10\text{ns}} \approx 66.7 \text{ MIPS}$$

Q Given Instruction Mix

50% ALU

30% Load

20% Branch

load stall $\rightarrow 2$ cycle

Branch penalty = 3 cycles

predictor accuracy = 80%

CPI ??

$\rightarrow$

Average load stall $\rightarrow 30\% \times 2$

0.6

Branch misprediction $\rightarrow 20\% \times \overset{\text{mispredict}}{20\%} \times 3$

0.12

$$CPI_{\text{effective}} = CPI_{\text{ideal}} + \text{Avg stall cycles}$$

$$= 1 + (0.6 + 0.12)$$

$$= 1.72$$

Q 5 stage pipeline

Branch mispredict penalty 3 cycles

Branch frequency 20%

predictor accuracy 90%

effective CPI ??

$\rightarrow$ Avg Branch stalls $\rightarrow \overset{\text{frequency mispredict}}{20\%} \times \overset{\text{frequency mispredict}}{10\%} \times 3$

$\rightarrow 0.06$

$$CPI_{\text{eff}} = 1 + 0.06$$

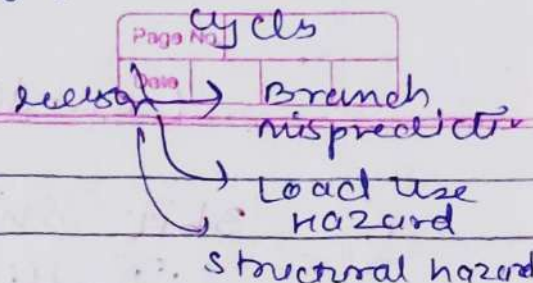
$$= 1.06$$

why not 30%
Bco 2 in correct
predictor then only
stalls introduced.

ideal CPI = 1

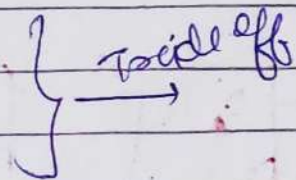
effective CPI = 1.8

$$1.8 = 1 + 0.8 \text{ stalls}$$



hardware cost of Forwarding Unit

- ✓ Extra MUX
- ✓ Wires
- ✓ Comparators



Area ↑↑

Power ↑↑

performance ✓

when stall is (unavoidable)

→ when required data or resource is not yet available & cannot be bypassed

Load use hazard

①

lw x_1 , $0(x_2)$

add x_3 , x_1 , x_4

→ Data memory content will be stored in L1R only after MEM stage

Forwarding unit cannot provide that data

therefore stall is necessary in this condition

②

same single memory access at same time by 2 different instructions.
∴ Need stall

not
only
stalls
introduces

Q. $CPI = 1$ does it mean CPU is fastest??

→ NO, BECOZ CPU ^{Total} execution time for N instruction depends on

$$\text{Total execution time} = N \times CPI \times \text{clock period}$$

CPU 1	CPU 2	CPU 3
$N = 1 \text{ Million}$	$N = 1 \text{ million}$	$N = 500K$
$CPI = 1$	$CPI = 2$	$CPI = 3$
$\text{clock} = 500 \text{ MHz}$ (2 ns)	$\text{clock} = 2 \text{ GHz}$ $= 0.5 \text{ ns}$	$\text{clock} = 2 \text{ GHz}$

$\begin{aligned} \text{Total CPU execution time} &= 1 \times 10^6 \times 1 \times 2 \times 10^{-9} \\ &= 1 \text{ ms} \end{aligned}$	$\begin{aligned} &= 1 \times 10^6 \times 2 \times 0.5 \times 10^{-9} \\ &= 1 \text{ ms} \end{aligned}$	$\begin{aligned} &= 500 \times 10^3 \times 3 \times 2 \times 10^{-9} \\ &= 0.7 \text{ ms} \end{aligned}$
--	--	--

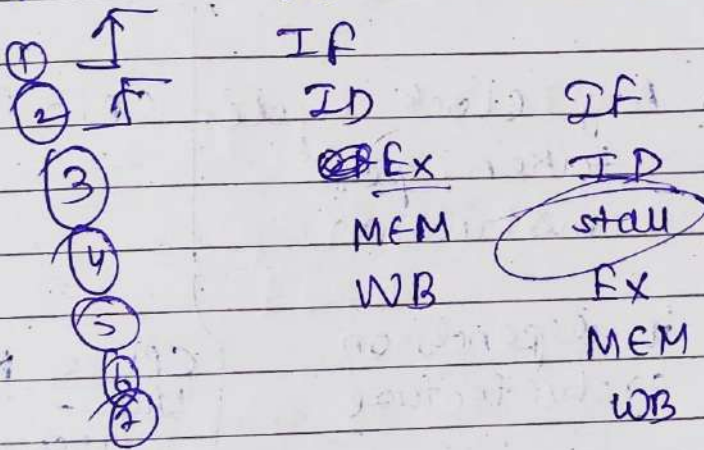
fastest

Q How many stalls for

LW: $R_1, (R_2)$

ADD R_3, R_1, R_4

if a pipelined CPU



solution to avoid stall

✓ compiler optimization
insert an independent instruction b/w
LW & ADD

Q If 5 stage Pipelined
for 10 instruction

$$\begin{aligned} \rightarrow \text{Total cycles} &= (k + n - 1) \\ &\quad \downarrow \quad \quad \downarrow \\ &\quad \text{1st instruction} \quad \text{remaining} \\ &\quad \text{cycles} \\ &= (5 + 10 - 1) \\ &= 14 \text{ cycles} \end{aligned}$$

$\rightarrow$ If 2 stalls are introduced per instruction

$$\begin{aligned} \text{Total cycles} &= (k + n - 1) + 2 \times (10) \\ &= (5 + 10 - 1) + 20 \\ &= 34 \text{ cycles} \end{aligned}$$

$\rightarrow$ if 30% branch instructions. $\rightarrow$ CPI increase??

pipeline
cycle time

↓
cum
hazards



Operating freq. of CPU ↑↑

Throughput ↑↑

Speedup ratio ↑↑ if hazards appear

Page No.	
Date	

No

Q given

Non pipelined

pipelined

CPI = 5 cycles

CPI = 1

Instruction = 1,000,000

freq = 1 GHz

speedup ??

$$\text{Speedup} = \frac{T_n}{T_p}$$

$$= \frac{1,000,000 \times (5 \times (1 \times 10^9 \text{ ns}))}{(k+n-1)t_p}$$

CPI × CPU cycle time

5 1,000,000

$$n \geq k-1$$

if stages are balanced

$$t_{\text{stage}} = k t_p$$

time taken to perform 1 instruction

$$= \frac{k t_p}{n t_p} \times \frac{n t_n}{(k+n-1)t_p}$$

$$= \frac{k t_n}{k t_p}$$

$$= \frac{t_n}{t_p}$$

$$S_{\text{ideal}} = \frac{k t_p}{t_p} = k = 5$$

Q

CPU 1 CPU 2 CPU 3

4 stage Pipe line 4 stage 5 stage Pipeline

$t_p = 2ns$ $t_p = 1.5ns$ $t_p = 1ns$

Which processor has highest f_{clock} ?

$$f_{max} = \frac{1}{T_{clk}}$$

✓ CPU 3

Q

Pipe line system ① → 800, 500, 400, 300

IF ID EX MEM

$$\text{Throughput} = \frac{1}{800ps} = \frac{1}{8 \times 10^{-10}}$$

if

800, 500, 400, 300

600 350

$$t_p = 600ns \text{ (new)}$$

$$\text{Throughput} = \frac{1}{600ps} = \frac{1}{6 \times 10^{-10}} = 1.67 \text{ GHz}$$

so it increases

Q

Each stage takes 10ns

$$\text{Throughput} = \frac{1}{10 \times 10^{-9}} = 100 \text{ million} \times 10^6$$

instructions per sec.

5 stage Pipeline System

$$t_{s1} = 8ns$$

$$t_{s2} = 10ns$$

$$t_{s3} = 7ns$$

$$t_{s4} = 9ns$$

$$t_{s5} = 11ns$$

800 tasks.

→ Not balanced pipeline system

→ Total processing time for 800 tasks

$$T_{\text{ex}} = (5 + 800 - 1) \times 11ns$$

$$= 8.844\mu s$$

↑ pipeline cycle time

→ $\text{Speedup} = \frac{t_{\text{non-p.}}}{t_{\text{pip. ex.}}} = \frac{n \times t_n}{T_{\text{ex}}}$

$$t_{\text{non-p.}} = t_{s1} + t_{s2} + t_{s3} + t_{s4} + t_{s5}$$

$$= 8 + 10 + 7 + 9 + 11$$

$$= 36ns$$

$$= 800 \times 36ns$$

$$= 8.844\mu s$$

$$= 4.07$$

$$T_{\text{exe pipeline}} = (k + (n-1)) t_p$$

$\downarrow$
 first instr. ka cycles

$\downarrow$
 remaining instr.

Q pipeline efficiency = $\frac{\text{Speedup (achieved)}}{\text{Sideal}}$

$$\text{Sideal} = \frac{t_n}{t_p} \quad \text{Sachieved} = \frac{n \times t_n}{(n + k - 1 + n \times t_{\text{stage}}) t_p}$$

Pipeline operating at 400 MHz has a speedup factor of 6 with 70% Efficiency
 $k??$

$$\Rightarrow 0.7 = \frac{6}{\text{Sideal}}$$

$$\frac{t_n}{t_p} = \text{Sideal} = 8.571$$

if stages are balanced $t_n = k t_p$
 $k = 8.571$

if stages are unbalanced
 $t_n < k t_p$
 $k > 8.571$

Q CPU 1 $\rightarrow f_{clk} = 1 \text{ GHz}$
 CPU 1 cycle time = $\frac{1}{1 \times 10^9}$

CPU 2? f_{clk} ?

$$t_{p2} = (1 - 0.25) t_{p1}$$

$$\frac{N \times (20\% + 100\%)}{f_{clk2}} = 0.75 \times \frac{N \times CPI \times f_{clk1}}{f_{clk1}}$$

$$f_{clk2} = \frac{0.75 \times 1 \text{ GHz}}{1.02}$$

$$\frac{120\%}{100}$$

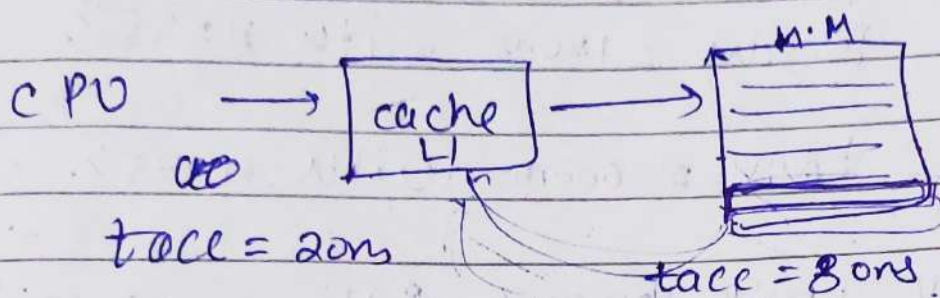
Q

ADD	R5, R0, R1	, R5 ← R0 + R1
MUL	R6, R2, R5	; R6 ← R2 * R5
SUB	R5, R3, R6	; R5 ← R3 - R6
DIV	R6, R5, R4	; R6 ← R5 / R4
STORE	R6, X	; BX ← R6

RAW = 4
dependencies

WAP = 2

WAW = 1



Hit ratio = 50%

CPU accessing cache & M.M simultaneously

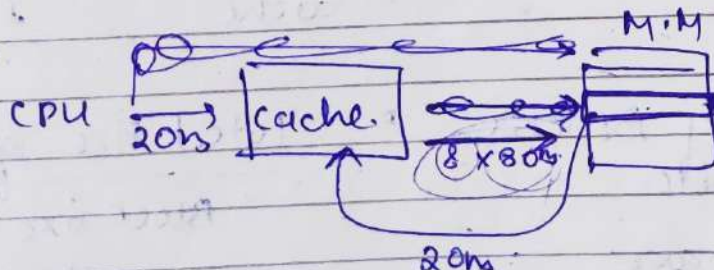
$$T_{\text{avg}} = H \times t_{\text{cache}} + (1-H) \left(\frac{t_{\text{block}}}{t_{\text{cache}}} + t_{\text{cache}} \right)$$

$$= 0.5 \times 20\text{ns} + (1-0.5) \left(\frac{8 \times 80\text{ns}}{20\text{ns}} + 20\text{ns} \right)$$

If Hierarchical

$$T_{\text{avg}} = H \times t_{\text{cache}} + (1-H) \left(t_{\text{cache}} + \frac{t_{\text{mm}}}{t_{\text{cache}}} \right)$$

$$= 0.5 \times 20\text{ns} + (1-0.5) \left(20\text{ns} + \frac{8 \times 80\text{ns}}{20\text{ns}} \right)$$



with
Block transfers

Q $t_{cache} = 120ns$ with $\eta = 85\%$.

$t_{mem} = 600ns$ with $\eta = 75\%$. $\rightarrow$ if $MHS = 25\%$.

if write through policy $\rightarrow$ write hit

Avg ??
 if 75% are ^{for} read operation
 25% write operation

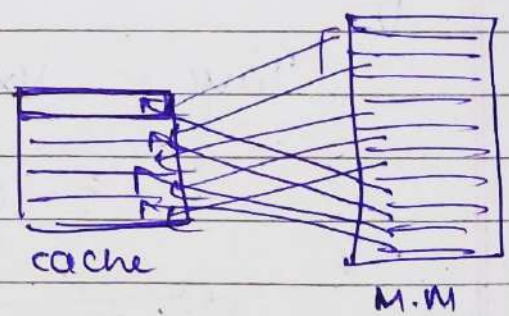
$\rightarrow$ Total Avg = $75\% \times T_{read} + 25\% \times T_{write}$

avg time taken by CPU to read dataword
 $T_{read} = (0.85) \times 120ns + (1-0.85) 600$

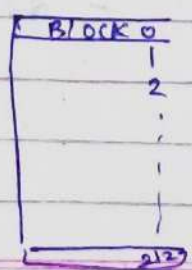
$T_{avg\ write} = \max(t_{cache}, t_{mem})$

avg time taken by CPU to modify dataword = 600ns

Q Direct cache Mapping



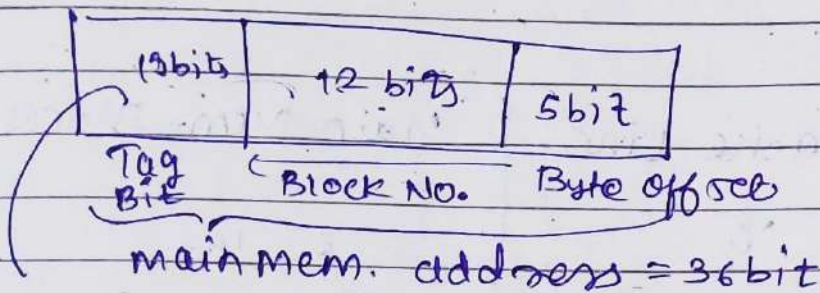
$\rightarrow$ No. of Blocks in Cache = $\frac{\text{Total size of cache}}{\text{Block size}}$



$= \frac{128 \times 1024}{32}$
 $= 2^{12}$

→ Block size = 32 Bytes

$$\log_2(32) = \underline{5} \text{ bits need for Byte offset}$$

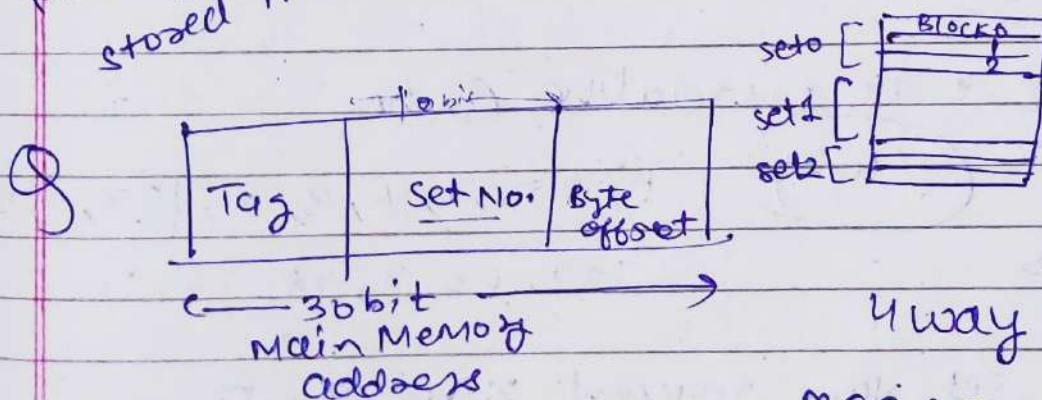


$$36 - 17$$

Tag directory size = $\frac{\text{No. of bits in Tag field}}{\text{No. of cache blocks}} \times \text{No. of cache blocks}$

Helps in identifying which Main Mem Block is stored in Cache Block

$$= 19 \times 2^{12} \text{ Bits}$$



$$\text{No. of sets} = \frac{2^{12}}{4} = 2^{10} = 1024$$

for addressing the set

Q cache Mem. $\rightarrow$ 8 ~~Blocks~~ Direct Mapping

Memory Block request

3, 5, 2, 8, 0, 63, 9, 16, 20, 17

$\rightarrow$ ~~find~~

cache Line	Main Mem Block
0	8, 0, 16
1	17
2	2
3	3
4	20
5	5
6	
7	63

$$\frac{\text{request block}}{8} = \text{Block Number}$$

$$\begin{array}{r} 0 \\ 8 \overline{) 3} \\ \underline{0} \\ 3 \end{array}$$

$$\begin{array}{r} 0 \\ 8 \overline{) 5} \\ \underline{0} \\ 5 \end{array}$$

$$\begin{array}{r} 0 \\ 8 \overline{) 2} \\ \underline{0} \\ 2 \end{array}$$

$$\begin{array}{r} 7 \\ 8 \overline{) 63} \\ \underline{56} \\ 7 \end{array}$$

Q 4 set associative Cache
4 way

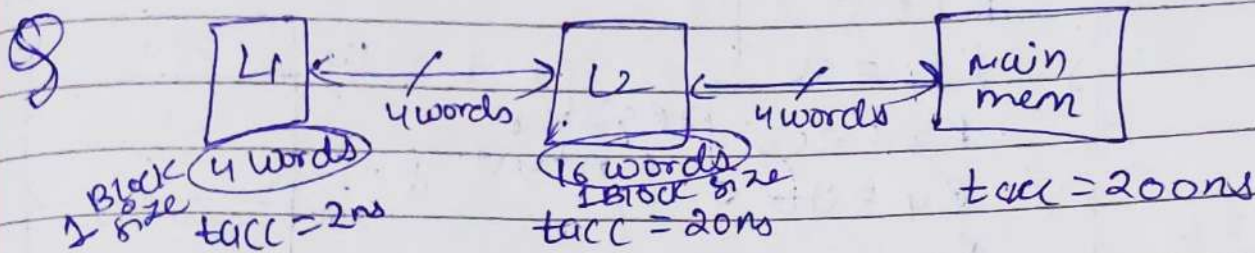
0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73

18 $\rightarrow$ conflict miss

Set No. = $\frac{\text{requested Block}}{\text{No. of sets}}$

set0	48 4 32	8 216 92
set1	1 129	133 73
set2		
set3	255 155 159	3 63

$$\begin{array}{r} 63 \\ 4 \overline{) 255} \\ \underline{240} \\ 15 \\ \underline{12} \\ 3 \end{array}$$



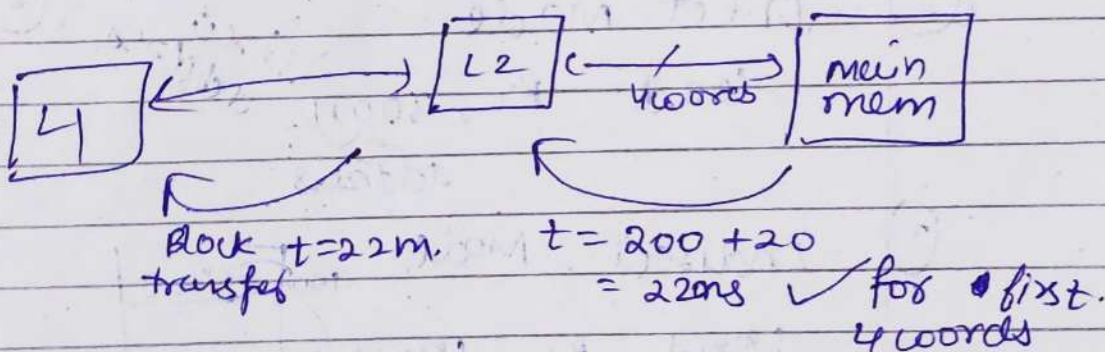
if Miss in L1 then hit in L2
Block is transferred from L2 to L1

Time ??
for transfer.

NO hit ratio
NO miss ratio
given

$$20ns + 2 = 22ns$$

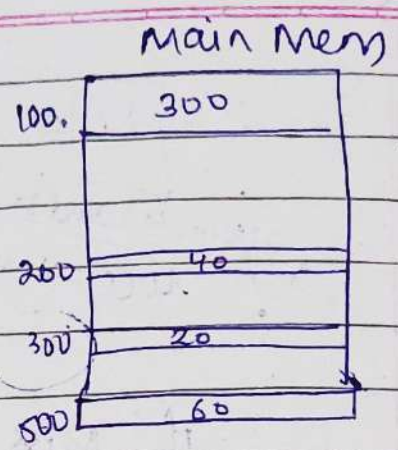
if $\left. \begin{matrix} L1 \\ L2 \end{matrix} \right\}$ Both miss



$$t = 4 \times 220 = 880ns \text{ for } 16 \text{ words}$$

Total time to transfer
from Main mem to L2 then to L1 = 880 + 22 = 902ns

$R_1 = 0$
 $R_2 = 50$
 $R_3 = 200$
 $R_4 = 90$ } register contents
 $PC = 100$
 Base add = 300



Load R_1 , operand

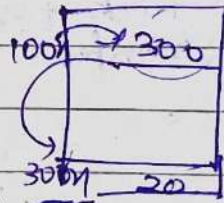
① Immediate Addressing mode
operand: 20 (value)

load R_1 , #20

② Direct mode
load R_1 , 300H
Direct address
address

③ Index Register mode

$$EA = \text{Regist} + \text{Base add.}$$

③ Indirect Mode
load R_1 , 100H
EA


$$R_1 = \text{mem}[EA]$$

④ Register Mode
load R_1 , R_2 $(R_1 = 50)$

⑤ Register indirect mode
load R_1 , # R_3
EA

Q A

$$N \text{ instruction execution time} = N \times CPI \times \text{CPU cycle time}$$

$$= \frac{N \times CPI}{f_{clk}}$$

jitna tezi se
CPU operate karega
utlaab instruction
execute jaldi
hoge, time
kam lagze

depends on CPI

✓ Total No. of instruction

✓ f_{clk} (cpu cycle time)

CPI → cycle per instruction
depends on Processor type $= \frac{\text{Total cycles}}{\text{instruction count}}$

Q CPU 1 CPU 2
Both receive N instructions
 $f_{clk} = 1 \text{ GHz}$ takes 25% less time than CPU 1
But 20% more CPI

$$t_{cpu2} = t_{cpu1} (1 - 25\%)$$

$$\frac{N \times CPI (1 + 20\%)}{f_{clk2}} = \frac{N \times CPI \times (1 - 0.25)}{f_{clk1}}$$

CPU cycle time $\rightarrow$ depends on architecture

single cycle = 1 cycle $\rightarrow$ 1 instruction executed

Pipelined $\Rightarrow$ 5 cycles $\rightarrow$ 1 instruction executed

	N	CPI	clock
CPU 1	1 million	1	500MHz $= 5 \times 10^8$
CPU 2	1 million	2	2GHz $= 2 \times 10^9$
CPU 3	500K	3	2GHz

which is fastest?

We can't conclude that

if $CPI = 1$: CPU 1 is faster

also by clock frequency : CPU 2 & CPU 3

operating at faster $\therefore$ so they are faster.

$$\text{CPU 1} \quad \text{Total execution time} = \frac{1 \times 10^6 \times 1 \times 1}{5 \times 10^8} = 0.2 \times 10^{-2} = 2 \text{ms}$$

$$\text{CPU 2} \quad = \frac{1 \times 10^6 \times 2 \times 1}{2 \times 10^9} = 10^{-3} = 1 \text{ms}$$

$$\text{CPU 3} \quad = \frac{500 \times 10^3 \times 3 \times 1}{2 \times 10^9} = 0.75 \text{ms}$$

fastest

Pipelined Based

Q

	K	tp
CPU 1	4 stage	2ns
CPU 2	4	1.5ns
CPU 3	5	1ns

Which CPU
~~fastest?~~
has highest
clock
freq?

$$t_{\text{pipeline CPU}} = (K + n - 1) t_p$$

total
stages
No. of insts
slowest stage time

Execution time
for N instruction

"K" stages
se confuse
nhi hona
hai

CPU 1

$$f = \frac{1}{2\text{ns}}$$

$$= 0.5\text{GHz}$$

CPU 2

$$f = \frac{1}{1.5\text{ns}}$$

$$=$$

CPU 3

$$f = \frac{1}{1\text{ns}}$$

$$= 1\text{GHz}$$

Q

K1	K2	K3	K4
800ns	600ns	400ns	300ns

Throughput $\rightarrow$ No. of instructions
completed per unit
time.

$$= \frac{1}{t_p (\text{slowest stage})}$$

$$= \frac{1}{800\text{ns}}$$

$$= \frac{n}{(K + n - 1) t_p}$$

$$n \gg K - 1$$

$$= \frac{n}{t_p}$$

Q 5 Stage pipelined system

$$t_{s1} = 8 \text{ ns}$$

$$t_{s2} = 10 \text{ ns}$$

$$t_{s3} = 7 \text{ ns}$$

$$t_{s4} = 9 \text{ ns}$$

$$t_{s5} = 11 \text{ ns}$$

$$N = 800 \text{ tasks}$$

Speedup ?

$$\rightarrow \text{Speedup} = \frac{t_{\text{nonpipelined}}}{t_{\text{pipelined}}} = \frac{N \times t_n}{(k+n-1) t_p}$$

$$t_n = \sum_{i=1}^k t_{si}$$

$$= \frac{800 \times (8 + 10 + 7 + 9 + 11)}{(5 + 800 - 1) \times 11 \text{ ns}}$$

→ if Balanced system

$$t_n = k t_p$$

$$= t_{s1} + t_{s2} + \dots$$

$$\frac{n t_n}{(k+n-1) t_p} = \frac{n t_n}{n t_p}$$

$$n > k-1 \Rightarrow \frac{k t_p}{t_p}$$

Ideal $\approx K$

if unbalanced

$$t_n < k t_p$$

$$S_0 = \left(\frac{t_n}{t_p} \right)$$

Q

32 bit CPU support of 128 MB Memory

To make it word addressable



4 Bytes = 32 bit data

CPU process 32 bit data at a time

4 Bytes

KB $\rightarrow 2^{10}$
 MB $\rightarrow 2^{20}$
 GB $\rightarrow 2^{30}$

$$128 \text{ MB} = 128 \times 2^{20} \text{ MB}$$

$$= 2^7 \times 2^{20}$$

$$= 2^{27}$$

$$2^{27} = 2^n \times m$$

$$2^{25} \times 2^2 = 2^n \times 4 \text{ Bytes}$$

data width

 $2^{25} \rightarrow$ Address lines

$2^{25} \rightarrow$ No. of words in Mem
 (No. of cells)
 No. of memory location

Q Total 200 instructions are there in a program

How much memory consumed

$$\rightarrow \text{Instruction length (Bits)} \times \text{Total instructions}$$

$$\downarrow \qquad \qquad \downarrow$$

$$(32 \text{ bit}) \times 200$$

$$6400 \text{ bits}$$

$$\frac{6400}{8} = 800 \text{ Bytes}$$

Q pipeline has $\rightarrow 8$ stages

each stage $t_{s1} = t_{s2} = \dots = t_{s8} = 2.5 \text{ ns}$

Balanced pipeline system

$\frac{1}{n}$ pipelined execution time

$$\rightarrow (K + n - 1) t_p$$

remaining instructions

for 1st insnce

$$K t_p = 8 \times 2.5$$

$$= 20.0$$

$$= 20 \text{ ns}$$

pipeline Latency

to execute 1 instruction complete

second

output appears after

$$20 + 2.5$$

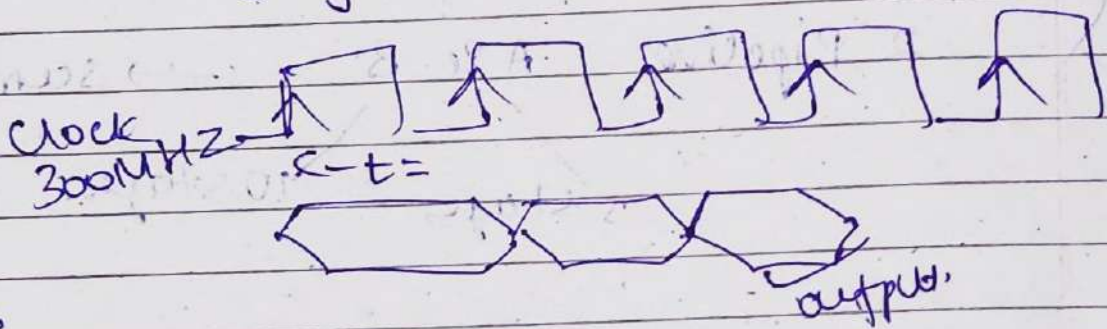
$$= 22.5 \text{ ns}$$

Q. pipeline latency = 10 cycles

Throughput $\rightarrow$ 1 item/cycle

$$\begin{aligned} \text{items in flight} &= \text{latency} \times \text{Throughput} \\ &= 10 \times 1 \\ &= 10 \end{aligned}$$

Q. pipeline produces 1 output after 3 cycles



$$\frac{1}{300 \times 10^6}$$

Throughput = no. of instructions completed per unit time

$$= \frac{n}{k + n - 1} \times \frac{1}{t_p} = \frac{1}{t_p}$$

output produce hone me kitna time liya (slowest stage time)

$$\begin{aligned} t_p &= 3T \\ &= 3 / (300 \times 10^6) = 1 \times 10^{-8} \text{ s} \\ &= 100 \text{ ns} \end{aligned}$$

$$\text{Throughput} = 100 \text{ M ops/sec}$$